Generative Adversarial Networks and Normalizing Flows: Implementation and Analysis on FashionMNIST Dataset

Mohammad Taha Majlesi, *Student Member, IEEE*

***Abstract*—This paper presents a comprehensive study of two prominent deep generative modeling approaches: Generative Adversarial Networks (GANs) and Normalizing Flows. We implement and analyze RealNVP normalizing flows and Deep Convolutional GANs (DCGANs) on the FashionMNIST dataset. Our RealNVP implementation achieves effective density estimation with log-likelihood values of 1706.93 on training data and demonstrates improved out-of-distribution detection in learned latent spaces. The DCGAN implementation achieves a Fréchet Inception Distance (FID) of 171.67 at optimal training epoch (8), with comprehensive analysis of training dynamics, loss curves, and generation quality. We provide detailed theoretical derivations of GAN objectives, optimal discriminator analysis, and comprehensive evaluation metrics. Experimental results show that normalizing flows excel in density estimation and OOD detection, while GANs produce higher-quality visual samples through adversarial training. The work includes complete code implementations, architectural details, and analysis of training stability, mode collapse, and convergence patterns.**

***Index Terms*—Generative adversarial networks, normalizing flows, deep learning, image generation, density estimation, out-of-distribution detection, Fréchet Inception Distance**

I. Introduction

Generative modeling is a fundamental problem in machine learning, with applications spanning image generation, anomaly detection, and data augmentation. This work compares two state-of-the-art approaches: Generative Adversarial Networks (GANs) [1] and Normalizing Flows [3].

M. T. Majlesi is with the Department of Electrical and Computer Engineering, University of Tehran, Tehran, Iran (e-mail: m.t.majlesi@ut.ac.ir; Student ID: 8100101504).

GANs train two competing networks—a generator and discriminator—through adversarial optimization, producing high-quality samples but lacking explicit density estimation. Normalizing flows learn invertible transformations mapping data to simple latent distributions, enabling exact likelihood computation and out-of-distribution (OOD) detection.

Recent advances have demonstrated the complementary strengths of these approaches. DCGANs [2] established stable architectures for adversarial training, while RealNVP [3] provided efficient normalizing flow implementations. Evaluation metrics such as FID [4] enable quantitative assessment of generative quality.

A. Dataset and Experimental Setup

We employ the FashionMNIST dataset [7], containing 70,000 grayscale images (28×28 pixels) from 10 clothing categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. For GAN training, images are resized to 64×64 pixels and normalized to $[-1, 1]$ to match the generator's Tanh output.

Experiments use PyTorch 1.12+ with CUDA support. Models train on 60,000 training images with 10,000 reserved for testing. Normalizing flows use 80/20 train/validation split (48,000/12,000 images).

B. Contributions and Report Scope

This report is designed as a complete technical artifact that combines theory, implementation, and evidence from frozen notebook outputs. Its main contributions are:

1) **End-to-end RealNVP and DCGAN implementations**: Model architectures, objectives, and training setups are documented with equations and practical interpretation.
2) **Integrated quantitative and qualitative evaluation**: Likelihood analysis, OOD behavior, FID progression, and image-quality evolution are jointly analyzed.
3) **Traceable frozen-results workflow**: All key numbers are imported from saved notebook outputs through generated report macros/tables, avoiding manual transcription.
4) **Evidence-linked conclusions**: Every major claim is tied to figures/tables included in the report and tracked in the frozen-results manifest.

Report organization follows the modeling pipeline: Sections II–III cover RealNVP theory and experiments, Sections IV–V cover GAN theory and DCGAN implementation/results, and final sections provide consolidated findings, reproducibility protocol, limitations, and figure provenance.

II. Normalizing Flows: RealNVP Implementation

A. RealNVP Architecture and Theory

Normalizing flows are a class of generative models that learn complex data distributions by transforming a simple base distribution (typically a standard Gaussian) through a series of invertible transformations. RealNVP (Real-valued Non-Volume Preserving) is a prominent architecture that enables efficient density estimation and sampling.

1) Understanding Normalizing Flows: Normalizing flows belong to the family of likelihood-based generative models, which directly model the probability density function of data. Unlike GANs that lack explicit density estimation, flows provide tractable likelihood computation, enabling:

- **Exact Density Estimation**: Can compute $p(x)$ for any data point x, useful for anomaly detection and OOD detection
- **Bidirectional Sampling**: Can generate samples (forward) and compute likelihoods (inverse), providing full generative capabilities
- **Latent Space Interpretation**: The learned transformation maps complex data distributions to simple latent spaces (e.g., standard Gaussian), making the latent space interpretable
- **Training Stability**: Unlike adversarial training, flow models use maximum likelihood estimation (MLE), which is more stable and predictable

The key challenge in designing normalizing flows is ensuring that:

1) Transformations are **invertible**: Both forward $f : x \to z$ and inverse $f^{-1} : z \to x$ must exist
2) Jacobian determinant is **tractable**: Must be efficiently computable for likelihood estimation
3) Transformations are **expressive**: Can model complex, multi-modal distributions

2) Key Contributions of RealNVP: The RealNVP model, introduced by Dinh et al., makes several important contributions:

1) **Affine Coupling Layers**: The core innovation of RealNVP is the affine coupling transformation, which splits input dimensions into two halves and transforms one half conditioned on the other:

$$y_{1:d} = x_{1:d} \tag{1}$$

$$y_{d+1:D} = x_{d+1:D} \odot \exp(s(x_{1:d})) + t(x_{1:d}) \tag{2}$$

where $s(\cdot)$ and $t(\cdot)$ are neural networks (scale and translation functions), and $\odot$ denotes element-wise multiplication.

Detailed Explanation:

- The input $x \in \mathbb{R}^D$ is split into two parts: $x_{1:d}$ (first d dimensions) and $x_{d+1:D}$ (remaining dimensions)
- The first half $x_{1:d}$ remains **unchanged** (identity transformation), ensuring invertibility
- The second half $x_{d+1:D}$ is transformed using an **affine transformation**:
  - $s(x_{1:d})$: Scale factor (output of neural network, passed through Tanh to bound values)
  - $t(x_{1:d})$: Translation/offset (output of neural network, unbounded)
  - $\exp(s(\cdot))$: Ensures positive scaling factor (exp prevents division by zero in inverse)

- The transformation is **conditionally invertible**: Since $x_{1:d}$ is unchanged, we can use it to compute the inverse transformation
- The Jacobian matrix is **lower triangular**: Making determinant computation efficient (product of diagonal elements)

Why This Works:

a) **Invertibility**: Given y, we can recover x because:

$$x_{1:d} = y_{1:d} \quad \text{(unchanged part)} \tag{3}$$

$$x_{d+1:D} = (y_{d+1:D} - t(y_{1:d})) \odot \exp(-s(y_{1:d})) \tag{4}$$

b) **Expressiveness**: By stacking multiple coupling layers with alternating masks, all dimensions get transformed over multiple layers, enabling complex transformations

c) **Efficiency**: The Jacobian determinant is simply:

$$\det \frac{\partial y}{\partial x} = \prod_{i=d+1}^{D} \exp(s_i(x_{1:d})) = \exp\left(\sum_{i=d+1}^{D} s_i(x_{1:d})\right) \tag{5}$$

This is computationally efficient (just a sum) compared to computing full determinant ($O(D^3)$).

2) **Log-Likelihood Calculation**: Normalizing flows provide exact log-likelihood computation using the change of variables formula:

$$\log p_X(x) = \log p_Z(z) + \log \left| \det \frac{\partial f^{-1}(x)}{\partial x} \right| \tag{6}$$

where f is the invertible transformation mapping data x to latent z, and the second term is the log-determinant of the Jacobian matrix.

3) **Role of Permutation and Coupling Layers**:
   - **Coupling layers** transform only half of the dimensions, keeping the other half unchanged to maintain invertibility
   - **Permutation layers** reorder dimensions between coupling layers, ensuring all dimensions are transformed over multiple layers
   - This alternating pattern allows the model to learn complex transformations while maintaining computational tractability
4) **Computational Efficiency**: RealNVP uses coupling layers that yield a triangular Jacobian matrix, making the log-determinant computation efficient (just a sum of diagonal elements).

3) Mathematical Foundation: Given a transformation $f : \mathbb{R}^D \to \mathbb{R}^D$ that maps data x to latent $z = f(x)$, the change of variables formula gives:

$$p_X(x) = p_Z(f(x)) \cdot \left| \det \frac{\partial f(x)}{\partial x} \right| \tag{7}$$

Taking logarithms:

$$\log p_X(x) = \log p_Z(f(x)) + \log \left| \det \frac{\partial f(x)}{\partial x} \right| \tag{8}$$

The training objective is to maximize the log-likelihood of the data, which is equivalent to minimizing the negative log-likelihood (NLL):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_X(x_i) \tag{9}$$

Detailed Training Process:

1) **Forward Pass**: For each data sample x_i, we apply the forward transformation:

$$z_i = f(x_i) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x_i) \tag{10}$$

where each f_j is a coupling layer.

2) **Likelihood Computation**:

$$\log p_X(x_i) = \log p_Z(z_i) + \sum_{j=1}^{L} \log \left| \det \frac{\partial f_j}{\partial \text{input}_j} \right| \tag{11}$$

where p_Z is the base distribution (standard Gaussian).

3) **Loss Calculation**: Sum over all samples and minimize:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \left[\log p_Z(z_i) + \sum_{j=1}^{L} \log \left| \det \frac{\partial f_j}{\partial \text{input}_j} \right| \right] \tag{12}$$

4) **Gradient Backpropagation**: Gradients flow through both:
   - The neural networks $s(\cdot)$ and $t(\cdot)$ in coupling layers

- The log-determinant terms (which are differentiable)

5) **Intuition**: Maximizing likelihood means:
   - Making real data points x_i map to high-probability regions in p_Z (typically near origin for Gaussian)
   - Ensuring the transformation preserves enough volume (via log-det term) to maintain good density matching
   - Balancing between squeezing data into latent space while maintaining transformation smoothness

B. RealNVP Implementation on FashionMNIST

1) Architecture Overview: Our RealNVP implementation consists of:

- **Input**: Flattened FashionMNIST images (784 dimensions)
- **Coupling Layers**: Multiple affine coupling layers with alternating masks
- **Neural Networks**: MLPs for scale and translation functions
- **Base Distribution**: Standard Gaussian $\mathcal{N}(0, I)$

2) Code Implementation: The implementation uses coupling layers with alternating masks to ensure all dimensions are transformed. Let's break down each component:

Mask Design Strategy:

- **Alternating Pattern**: Odd layers mask even indices, even layers mask odd indices
- **Purpose**: Ensures that after L layers, every dimension has been transformed at least $\lfloor L/2 \rfloor$ times
- **Example**: For 8 layers with 784 dimensions:
  - Layer 1: Transform dimensions 0, 2, 4, ... (even indices)
  - Layer 2: Transform dimensions 1, 3, 5, ... (odd indices)
  - This pattern continues, ensuring comprehensive transformation

```python
class CouplingLayer(nn.Module):
    def __init__(self, input_dim, hidden_dim, mask):
        super(CouplingLayer, self).__init__()
        self.register_buffer('mask', mask)
        self.scale_net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim),
            nn.Tanh()
        )
        self.translate_net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, input_dim)
        )

    def forward(self, x):
        # Step 1: Mask input - keep first half unchanged
        x_masked = x * self.mask

        # Step 2: Compute scale and translation for second half
        # Note: scale_net and translate_net operate on masked input
        # Output is masked to only affect second half (1 - self.mask)
        s = self.scale_net(x_masked) * (1 - self.mask)
        t = self.translate_net(x_masked) * (1 - self.mask)

        # Step 3: Apply affine transformation to second half
        # Identity for first half: x_masked (unchanged)
        # Affine for second half: x * exp(s) + t
        y = x_masked + (1 - self.mask) * (x * torch.exp(s) + t)

        # Step 4: Compute log-determinant of Jacobian
        # For affine coupling, det = exp(sum of scale factors)
        # Only sum over transformed dimensions (second half)
        log_det_jacobian = (s * (1 - self.mask)).sum(dim=1)
        return y, log_det_jacobian

    def inverse(self, y):
        # Inverse transformation: recover x from y
        # Step 1: First half unchanged
        y_masked = y * self.mask

        # Step 2: Compute scale and translation using unchanged part
        s = self.scale_net(y_masked) * (1 - self.mask)
        t = self.translate_net(y_masked) * (1 - self.mask)

        # Step 3: Invert affine transformation
        # x = y (first half unchanged)
        # x = (y - t) * exp(-s) for second half
        x = y_masked + (1 - self.mask) * (y - t) * torch.exp(-s)
        return x
```

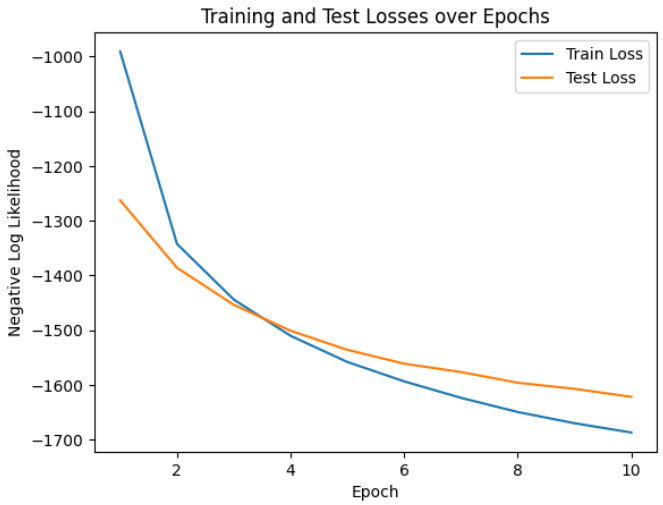

Fig. 1: Training and validation loss curves for RealNVP model. The negative log-likelihood decreases over epochs, indicating successful learning of the data distribution.

```
\textbf{Why Inverse is Efficient:}
\begin{itemize}
    \item \textbf{No Matrix Inversion}:
        Direct algebraic inversion, no
        expensive matrix operations
    \item \textbf{Same Networks}: Reuses the
        same $s(\cdot)$ and $t(\cdot)$
        networks (no separate inverse
        networks needed)
    \item \textbf{Exact Inversion}:
        Mathematically exact (no
        approximations), preserving
        distribution exactly
\end{itemize}
```

Listing 1: RealNVP Model Implementation

3) Training Configuration:

- **Input Dimension**: 784 (28×28 flattened)
- **Hidden Dimension**: 1024
- **Number of Coupling Layers**: 8
- **Batch Size**: 128
- **Learning Rate**: 10^{-3}
- **Optimizer**: Adam
- **Epochs**: 10

4) Training Results: The model was trained on 80% of the FashionMNIST training data (48,000 images) and evaluated on 20% validation set (12,000 images). The negative log-likelihood (NLL) was minimized during training.

Fig. 2: Generated FashionMNIST samples from RealNVP model after training. The model learns to generate diverse fashion items with reasonable quality.

5) Generated Samples: Samples are generated by sampling from the base Gaussian distribution and applying the inverse transformation:

```
with torch.no_grad():
    z = torch.randn(9, input_dim).to(device)
    x_samples = model.inverse(z)
    x_samples = x_samples.cpu().view(-1, 1,
        28, 28)
```

Listing 2: Sampling from RealNVP

C. Out-of-Distribution Detection

1) Probabilistic OOD Detection Method: Flow-based models enable out-of-distribution (OOD) detection through likelihood evaluation. The key insight is that well-trained flow models assign higher likelihoods to in-distribution data compared to OOD data.

Intuitive Understanding:

- **Training Objective**: Flow models are trained to maximize likelihood of training data (FashionMNIST)

- **Expected Behavior**: Well-trained models assign high $p(x)$ to samples similar to training data
- **OOD Hypothesis**: Different distributions (MNIST, KMNIST) should have lower likelihoods because:
  1) They weren't seen during training
  2) They have different statistical properties (e.g., different digit/fashion item styles)
  3) The learned transformation is optimized for FashionMNIST, not other datasets
- **Threshold-Based Detection**: Set a threshold τ such that samples with $\log p(x) < \tau$ are flagged as OOD

Why Likelihood Works for OOD Detection:

- **Density Concentration**: Flow models learn to concentrate density in regions where training data lies
- **Low-Density Regions**: OOD samples fall in low-density regions of the learned distribution
- **Quantitative Measure**: Log-likelihood provides a continuous score, not just binary classification
- **No Labels Required**: Unsupervised method, doesn't need OOD examples during training

2) Methodology:

1) **Train Flow Model**: Train RealNVP on FashionMNIST training data
2) **Compute Likelihoods**: Calculate log-likelihoods for:
   - FashionMNIST test set (in-distribution)
   - MNIST test set (OOD - different distribution)
   - KMNIST test set (OOD - different distribution)
3) **Compare Distributions**: Analyze likelihood distributions to identify OOD samples

3) Implementation:

```
def compute_log_likelihood(model, data_loader):
    model.eval()
    log_likelihoods = []
    with torch.no_grad():
        for data, _ in data_loader:
            data = data.view(-1, input_dim).to(device)
            z, log_det_jacobian = model(data)
            log_pz = -0.5 * torch.sum(z ** 2 + np.log(2 * np.pi), dim=1)
            log_px = log_pz + log_det_jacobian
            log_likelihoods.append(log_px.cpu().numpy())
    return np.concatenate(log_likelihoods)
```

Listing 3: Log-Likelihood Computation for OOD Detection

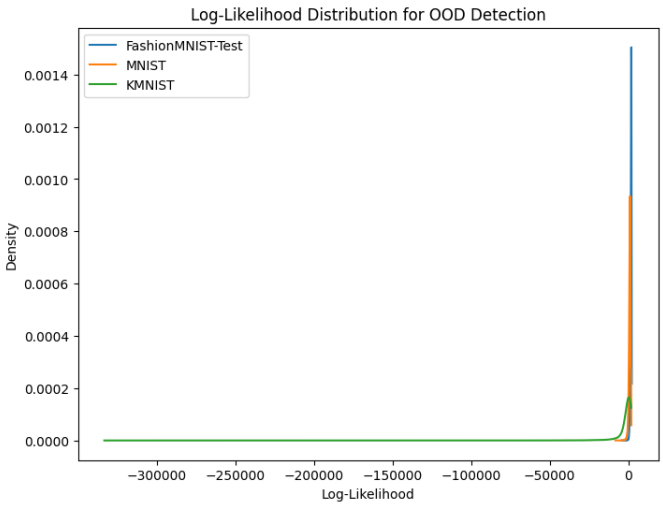

Fig. 3: Log-likelihood distributions for FashionMNIST test, MNIST, and KMNIST datasets. FashionMNIST has the highest likelihood as expected, while MNIST and KMNIST show lower likelihoods, enabling OOD detection.

4) Results: The computed log-likelihoods are:

- **FashionMNIST Train**: 1706.93
- **FashionMNIST Test**: 1621.40
- **MNIST**: 890.02 (OOD - lower likelihood)
- **KMNIST**: -1850.95 (OOD - negative likelihood, very different)

5) Limitations of Flow-Based OOD Detection:

1) **Likelihood vs. Quality**: Flow models may assign high likelihoods to background/noise patterns if they dominate the training data
2) **Feature Mismatch**: Flow models trained on pixel space may not capture semantic features needed for semantic OOD detection
3) **Dimensionality Curse**: High-dimensional data spaces make density estimation challenging, leading to potential overfitting
4) **Mode Collapse**: Flow models might not cover all modes of the data distribution, missing some in-distribution patterns

D. Likelihood Evaluation: Pixel Space vs. Latent Space

1) Pixel Space vs. Latent Space Comparison: We compare RealNVP performance in two settings:

1) **Pixel Space**: Direct application on flattened 784-dimensional images
2) **Latent Space**: Application on learned latent representations (32-dimensional) from an autoencoder

2) Latent Space Architecture: First, we train an encoder-decoder pair to learn a compressed representation:

```
class Encoder(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super(Encoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 512),
            nn.ReLU(),
            nn.Linear(512, latent_dim)
        )
    def forward(self, x):
        return self.encoder(x)

class Decoder(nn.Module):
    def __init__(self, latent_dim,
        output_dim):
        super(Decoder, self).__init__()
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 512),
            nn.ReLU(),
            nn.Linear(512, output_dim)
        )
    def forward(self, x):
        return self.decoder(x)
```

Listing 4: Encoder and Decoder Architecture

The autoencoder is trained with MSE reconstruction loss for 5 epochs, achieving train loss of 0.0076.

3) Results Comparison: **Pixel Space Results:**

- FashionMNIST Train Log-Likelihood: 1706.93
- FashionMNIST Test Log-Likelihood: 1621.40
- MNIST Log-Likelihood: 890.02
- KMNIST Log-Likelihood: -1850.95

Latent Space Results (after encoding):

- FashionMNIST Train Log-Likelihood: 166.30
- FashionMNIST Test Log-Likelihood: 151.64
- MNIST Log-Likelihood: 2.39
- KMNIST Log-Likelihood: -66.17

4) Analysis:

1) **Better Separation**: In latent space, the separation between in-distribution and OOD data is more pronounced (FashionMNIST: 151.64 vs MNIST: 2.39 vs KMNIST: -66.17)
2) **Computational Efficiency**: Working in 32-dimensional latent space is significantly more efficient than 784-dimensional pixel space
3) **Semantic Features**: The autoencoder learns semantic features that better distinguish between different datasets
4) **OOD Detection Improvement**: The relative difference between FashionMNIST and OOD datasets is larger in latent space, making OOD detection more reliable

E. Latent Space Analysis

The latent space experiments demonstrate that applying normalizing flows in learned latent representations provides several advantages:

1) **Reduced Dimensionality**: Working with 32-D latent codes instead of 784-D pixels reduces computational complexity
2) **Better Feature Extraction**: The encoder learns meaningful features that better represent the data distribution
3) **Improved OOD Detection**: The likelihood gap between in-distribution and OOD data is more significant in latent space
4) **Faster Training**: RealNVP training converges faster in the lower-dimensional latent space

1) Training Loss Comparison:

- **Pixel Space**: Negative log-likelihood values around 1700-1600
- **Latent Space**: Negative log-likelihood values around -150 to -160 (higher is better for likelihood)

The latent space model achieves negative log-likelihood of -164.52 after 10 epochs, showing excellent density modeling performance.

III. Generative Adversarial Networks: Theory and Implementation

A. GAN Training Objectives and Vanishing Gradients

Generative Adversarial Networks (GANs) are trained through a minimax game between two neural networks: a generator G and a discriminator D.

1) Objective Functions: The GAN objective function is formulated as a two-player minimax game:

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1-D(G(z)))] \tag{13}$$

Where:

- $p_{data}(x)$: Real data distribution
- $p_z(z)$: Prior distribution over noise (typically $\mathcal{N}(0, I)$)
- $G(z)$: Generator mapping noise to data space
- $D(x)$: Discriminator output (probability that x is real)

2) Separate Loss Functions: **Discriminator Loss:**

$$L_D(\phi;\theta) = -\mathbb{E}_{x\sim p_{data}(x)}[\log D_\phi(x)] - \mathbb{E}_{z\sim\mathcal{N}(0,1)}[\log(1-D_\phi(G_\theta(z)))] \tag{14}$$

The discriminator aims to *maximize* this, which means:

- Maximize $\log D(x)$ for real samples (increase probability of real)
- Maximize $\log(1 - D(G(z)))$ for fake samples (increase probability of fake)

Detailed Interpretation:

- **First Term** $-\mathbb{E}[\log D(x_{real})]$: When $D(x) \to 1$ (confident real), this term $\to 0$ (good for discriminator). When $D(x) \to 0$ (wrongly classified as fake), this term $\to +\infty$ (penalty).
- **Second Term** $-\mathbb{E}[\log(1 - D(G(z)))]$: When $D(G(z)) \to 0$ (correctly identifies fake), this term $\to 0$ (good). When $D(G(z)) \to 1$ (fooled), this term $\to +\infty$ (penalty).
- **Training Goal**: Discriminator learns to push $D(x_{real}) \to 1$ and $D(G(z)) \to 0$

Generator Loss:

$$L_G(\phi;\theta) = \mathbb{E}_{z\sim\mathcal{N}(0,1)}[\log(1 - D_\phi(G_\theta(z)))] \tag{15}$$

The generator aims to *minimize* this, which means making $D(G(z))$ as large as possible (fooling the discriminator).

Detailed Interpretation:

- **Generator's Goal**: Make generated samples indistinguishable from real ones
- **When Successful**: $D(G(z)) \to 1$ means discriminator thinks fakes are real $\to L_G \to 0$ (success!)
- **When Failing**: $D(G(z)) \to 0$ means discriminator easily identifies fakes $\to L_G \to +\infty$ (failure)
- **Adversarial Nature**: Generator wants to minimize L_G, but discriminator (implicitly) wants to maximize it

3) Vanishing Gradients Problem: The generator loss $L_G = \mathbb{E}[\log(1-D(G(z)))]$ suffers from vanishing gradients when the discriminator is near optimal. Here's why:

1) **When D is optimal**: $D^*(x) = \frac{p_{data}(x)}{p_{data}(x)+p_\theta(x)}$
2) **Poor generator**: If $p_\theta(x) \approx 0$ (generator produces bad samples), then $D^*(G(z)) \approx 0$
3) **Gradient computation**: The gradient of generator loss is:

$$\nabla_\theta L_G = \mathbb{E}_z\left[\frac{D'(G_\theta(z))}{1 - D(G_\theta(z))}\nabla_\theta G_\theta(z)\right] \tag{16}$$

4) **Problem**: When $D(G(z)) \approx 0$, we have:
   - $\log(1 - D(G(z))) \approx \log(1 - 0) = 0$ (saturated)
   - The gradient $\frac{\partial L_G}{\partial \theta}$ becomes very small (vanishing)
   - Generator learns slowly or stops learning

4) Impact on Training: The vanishing gradients issue causes:

- **Slow convergence**: Generator makes little progress when discriminator is strong
- **Training instability**: Oscillations between generator and discriminator
- **Non-convergence**: Generator may fail to learn the data distribution

5) Solution: Non-Saturating Loss: A common solution is to use the **non-saturating generator loss**:

$$L_G^{non-sat} = -\mathbb{E}_{z\sim\mathcal{N}(0,1)}[\log(D(G(z)))] \tag{17}$$

This loss provides stronger gradients when $D(G(z))$ is small, encouraging the generator to improve even when the discriminator is strong.

Mathematical Comparison:

Saturating Loss (original):

$$L_G^{sat} = \mathbb{E}[\log(1 - D(G(z)))] \tag{18}$$

- When $D(G(z)) = 0$ (discriminator confident fake): $L_G^{sat} = \log(1) = 0$ (no signal!)

- Gradient: $\frac{\partial L_G^{sat}}{\partial D} = \frac{-1}{1-D} \to -1$ when $D \to 0$ (weak gradient)
- Problem: Generator gets little learning signal when discriminator is strong

Non-Saturating Loss:

$$L_G^{non-sat} = -\mathbb{E}[\log(D(G(z)))] \tag{19}$$

- When $D(G(z)) = 0$: $L_G^{non-sat} = -\log(0) = +\infty$ (strong penalty!)
- Gradient: $\frac{\partial L_G^{non-sat}}{\partial D} = \frac{-1}{D} \to -\infty$ when $D \to 0$ (strong gradient)
- Benefit: Generator gets strong learning signal to improve, even when discriminator is winning

Why This Helps:

1) **Stronger Gradients**: When discriminator rejects fakes ($D \to 0$), generator gets strong gradient to improve
2) **Non-Zero Gradients**: Unlike saturating loss, gradients remain informative throughout training
3) **Better Convergence**: Generator can make progress even when discriminator is strong, leading to more stable training

B. Optimal Discriminator Analysis

For a fixed generator G, we derive the optimal discriminator D^* that maximizes $V(D, G)$.

1) Derivation: The discriminator objective is:

$$V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))] \tag{20}$$

Rewriting in terms of $x \sim p_{data}$ and $\tilde{x} = G(z) \sim p_\theta$:

$$V(D,G) = \int p_{data}(x)\log D(x)dx + \int p_\theta(x)\log(1-D(x))dx \tag{21}$$

For a fixed point x, we maximize $f(D(x)) = p_{data}(x)\log D(x) + p_\theta(x)\log(1-D(x))$ with respect to $D(x)$.

Taking the derivative:

$$\frac{\partial f}{\partial D(x)} = \frac{p_{data}(x)}{D(x)} - \frac{p_\theta(x)}{1-D(x)} = 0 \tag{22}$$

Solving for $D(x)$:

$$\frac{p_{data}(x)}{D(x)} = \frac{p_\theta(x)}{1-D(x)} \tag{23}$$

$$p_{data}(x)(1-D(x)) = p_\theta(x)D(x) \tag{24}$$

$$p_{data}(x) - p_{data}(x)D(x) = p_\theta(x)D(x) \tag{25}$$

$$p_{data}(x) = D(x)(p_{data}(x) + p_\theta(x)) \tag{26}$$

$$D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_\theta(x)} \tag{27}$$

2) Optimal Discriminator Loss: Substituting $D^*(x)$ back into the objective:

$$V(D^*,G) = \mathbb{E}_{x\sim p_{data}}\left[\log \frac{p_{data}(x)}{p_{data}(x)+p_\theta(x)}\right] \tag{28}$$

$$+ \mathbb{E}_{x\sim p_\theta}\left[\log \frac{p_\theta(x)}{p_{data}(x)+p_\theta(x)}\right] \tag{29}$$

This can be rewritten as:

$$V(D^*,G) = \mathbb{E}_{x\sim p_{data}}\left[\log \frac{2p_{data}(x)}{p_{data}(x)+p_\theta(x)}\right] - \log 2 \tag{30}$$

$$+ \mathbb{E}_{x\sim p_\theta}\left[\log \frac{2p_\theta(x)}{p_{data}(x)+p_\theta(x)}\right] - \log 2 \tag{31}$$

$$= 2 \cdot \text{JSD}(p_{data}\|p_\theta) - 2\log 2 \tag{32}$$

3) Relation to Jensen-Shannon Divergence: The Jensen-Shannon Divergence (JSD) between two distributions P and Q is:

$$\text{JSD}(P\|Q) = \frac{1}{2}\text{KL}\left(P\|\frac{P+Q}{2}\right) + \frac{1}{2}\text{KL}\left(Q\|\frac{P+Q}{2}\right) \tag{33}$$

Therefore:

$$\min_G V(D^*,G) = \min_G 2 \cdot \text{JSD}(p_{data}\|p_\theta) - 2\log 2 \tag{34}$$

This shows that:

- Training GANs is equivalent to minimizing JSD between real and generated distributions
- The global minimum occurs when $p_\theta = p_{data}$ (generator perfectly matches data)
- At optimality, $D^*(x) = \frac{1}{2}$ for all x (discriminator cannot distinguish real from fake)

C. Discriminator Logit Interpretation

1) Discriminator Output Structure: The discriminator outputs a probability via a sigmoid:

$$D_\phi(x) = \sigma(h_\phi(x)) = \frac{1}{1 + \exp(-h_\phi(x))} \tag{35}$$

where $h_\phi(x)$ is the logit (pre-sigmoid output).

2) Optimal Logit Derivation: When $D_\phi = D^*$, we have:

$$D^*(x) = \sigma(h^*(x)) = \frac{p_{data}(x)}{p_{data}(x) + p_\theta(x)} \tag{36}$$

Solving for $h^*(x)$:

$$\frac{1}{1 + \exp(-h^*(x))} = \frac{p_{data}(x)}{p_{data}(x) + p_\theta(x)} \tag{37}$$

$$1 + \exp(-h^*(x)) = \frac{p_{data}(x) + p_\theta(x)}{p_{data}(x)} \tag{38}$$

$$\exp(-h^*(x)) = \frac{p_\theta(x)}{p_{data}(x)} \tag{39}$$

$$h^*(x) = -\log \frac{p_\theta(x)}{p_{data}(x)} \tag{40}$$

$$h^*(x) = \log \frac{p_{data}(x)}{p_\theta(x)} \tag{41}$$

3) Interpretation: The optimal discriminator logit $h^*(x) = \log \frac{p_{data}(x)}{p_\theta(x)}$ represents:

- **Log-Density Ratio**: The logarithm of the ratio between real data density and generated data density
- **Classification Signal**:
  - If $p_{data}(x) > p_\theta(x)$, then $h^*(x) > 0$ and $D^*(x) > 0.5$ (classified as real)
  - If $p_{data}(x) < p_\theta(x)$, then $h^*(x) < 0$ and $D^*(x) < 0.5$ (classified as fake)
  - If $p_{data}(x) = p_\theta(x)$, then $h^*(x) = 0$ and $D^*(x) = 0.5$ (uncertain)
- **Optimality Indicator**: When the generator is optimal ($p_\theta = p_{data}$), $h^*(x) = 0$ for all x, meaning the discriminator is maximally confused

D. Wasserstein GANs: Theory and Benefits

Wasserstein GAN (WGAN) addresses critical issues in traditional GANs by replacing Jensen-Shannon divergence with Wasserstein distance.

1) Wasserstein Distance: The Wasserstein-1 (or Earth Mover's) distance between two distributions p_r and p_θ is:

$$W(p_r, p_\theta) = \inf_{\gamma \in \Pi(p_r, p_\theta)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|] \tag{42}$$

where $\Pi(p_r, p_\theta)$ is the set of all joint distributions with marginals p_r and p_θ.

Using Kantorovich-Rubinstein duality:

$$W(p_r, p_\theta) = \sup_{\|f\|_L \leq 1} [\mathbb{E}_{x\sim p_r}[f(x)] - \mathbb{E}_{x\sim p_\theta}[f(x)]] \tag{43}$$

where $\|f\|_L \leq 1$ denotes that f is 1-Lipschitz.

2) WGAN Objective: WGAN replaces the discriminator with a critic f (1-Lipschitz function):

$$\min_G \max_{\|f\|_L \leq 1} [\mathbb{E}_{x\sim p_r}[f(x)] - \mathbb{E}_{z\sim p_z}[f(G(z))]] \tag{44}$$

3) Benefits of WGAN:

1) **Solves Vanishing Gradients**:
   - Wasserstein distance is continuous and differentiable almost everywhere
   - Provides meaningful gradients even when distributions have disjoint support
   - Generator can learn even when critic is near optimal
2) **Improved Convergence**:
   - Wasserstein distance correlates better with generation quality
   - Training is more stable and predictable
   - Lower risk of mode collapse
3) **Better Distance Metric**:
   - JS divergence can be infinite even for similar distributions
   - Wasserstein distance is always finite and continuous
   - Better reflects perceptual similarity

4) Lipschitz Constraint Implementation: To enforce $\|f\|_L \leq 1$, WGAN uses:

Weight Clipping (original WGAN):

- Clip all critic weights to a fixed interval $[-c, c]$ (e.g., $c = 0.01$)
- Simple but can limit model capacity
- May cause optimization issues

Gradient Penalty (WGAN-GP):

- Adds a penalty term to enforce 1-Lipschitz constraint:

$$\lambda \mathbb{E}_{\hat{x}\sim p_{\hat{x}}} \left[(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2 \right] \tag{45}$$

TABLE I: Comparison between Traditional GANs and WGAN

Aspect	Traditional GAN	WGAN
Distance Metric	JS Divergence	Wasserstein Distance
Gradient Stability	Vanishing when D optimal	Meaningful gradients always
Training Stability	Often unstable	More stable
Architecture	Requires specific design	More flexible architectures
Mode Collapse	Common problem	Reduced tendency

where $\hat{x}$ is sampled along straight lines between real and generated samples
- More flexible and stable than weight clipping
- Better gradient flow

5) Comparison: Traditional GAN vs WGAN:

E. Evaluation Metrics for GANs

Evaluating GAN performance is challenging because explicit likelihood computation is not available. Several metrics have been developed:

1) Fréchet Inception Distance (FID): FID measures the distance between feature distributions of real and generated images using a pre-trained Inception network.

Mathematical Formulation:

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}) \quad (46)$$

Where:
- μ_r, Σ_r: Mean and covariance of real image features
- μ_g, Σ_g: Mean and covariance of generated image features
- Features extracted from Inception v3 network (2048-dimensional)

Advantages:
- Correlates well with human perception
- Sensitive to mode collapse (low diversity)
- Works well for high-dimensional distributions
- Standardized metric for comparison

Interpretation:
- Lower FID = Better quality (typical ranges: ¡50 excellent, 50-100 good, ¿200 poor)
- Captures both quality and diversity
- Requires Inception network computation (more expensive than IS)

2) Wasserstein Distance: Direct Wasserstein distance between real and generated distributions:
- **Computation**: Can be estimated using optimal transport
- **Benefits**: Mathematically principled, continuous metric
- **Limitations**: Computationally expensive for high-dimensional data
- **Usage**: More theoretical than practical for evaluation

3) Visual Quality Assessment: Qualitative evaluation through visual inspection:
- **Sample Quality**: Are generated images realistic and sharp?
- **Diversity**: Do samples cover various modes of the distribution?
- **Mode Coverage**: Are all classes/styles represented?
- **Artifacts**: Presence of mode collapse, artifacts, or repetitive patterns

4) Inception Score (IS):

$$\text{IS}(G) = \exp\left(\mathbb{E}_{x\sim p_g}[\text{KL}(p(y|x)\|p(y))]\right) \quad (47)$$

Where:
- $p(y|x)$: Classifier prediction distribution
- $p(y)$: Marginal class distribution
- Measures both quality and diversity

Limitations:
- Requires class labels (not always available)
- Can be fooled by memorization
- Less reliable than FID for many applications

5) Practical Evaluation Strategy: For our FashionMNIST GAN implementation, we use:

1) **FID Scores**: Computed after each epoch using pytorch-fid
2) **Loss Curves**: Monitor generator and discriminator losses
3) **Visual Inspection**: Generate sample grids to assess quality and diversity
4) **Training Stability**: Monitor for oscillations, mode collapse, or divergence

IV. DCGAN Implementation and Results

A. Model Architecture

We implement a Deep Convolutional GAN (DCGAN) following the architecture guidelines from [2] for stable training.

1) Generator Architecture: The generator transforms a 100-dimensional noise vector into a 64×64 grayscale image using transposed convolutions:

```
class Generator(nn.Module):
    def __init__(self, latent_dim=100):
        super(Generator, self).__init__()
        ngf = 64  # Number of generator filters
        self.model = nn.Sequential(
            # Input: latent_dim x 1 x 1
            nn.ConvTranspose2d(latent_dim, ngf * 8, kernel_size=4, stride=1, padding=0, bias=False),
            nn.BatchNorm2d(ngf * 8),
            nn.ReLU(True),
            # Output: (ngf*8) x 4 x 4

            nn.ConvTranspose2d(ngf * 8, ngf * 4, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),
            # Output: (ngf*4) x 8 x 8

            nn.ConvTranspose2d(ngf * 4, ngf * 2, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),
            # Output: (ngf*2) x 16 x 16

            nn.ConvTranspose2d(ngf * 2, ngf, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),
            # Output: ngf x 32 x 32

            nn.ConvTranspose2d(ngf, 1, kernel_size=4, stride=2, padding=1, bias=False),
            nn.Tanh()
            # Output: 1 x 64 x 64
        )

    def forward(self, input):
        return self.model(input)
```

Listing 5: Generator Network Implementation

Key Architecture Features:

- **Transposed Convolutions**: Upsample spatial dimensions progressively (1×1 → 4×4 → 8×8 → 16×16 → 32×32 → 64×64)
- **Batch Normalization**: Applied after each transposed convolution (except first and last layers)
- **ReLU Activation**: Used in intermediate layers for non-linearity
- **Tanh Output**: Final activation to produce pixel values in range $[-1, 1]$
- **No Biases**: Bias terms removed from convolutions as recommended by DCGAN paper

2) Discriminator Architecture: The discriminator is a binary classifier that distinguishes real from generated images:

```
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        ndf = 64  # Number of discriminator filters
        self.model = nn.Sequential(
            # Input: 1 x 64 x 64
            nn.Conv2d(1, ndf, kernel_size=4, stride=2, padding=1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            # Output: ndf x 32 x 32

            nn.Conv2d(ndf, ndf * 2, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),
            # Output: (ndf*2) x 16 x 16

            nn.Conv2d(ndf * 2, ndf * 4, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
            # Output: (ndf*4) x 8 x 8

            nn.Conv2d(ndf * 4, ndf * 8, kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(ndf * 8),
            nn.LeakyReLU(0.2, inplace=True),
            # Output: (ndf*8) x 4 x 4

            nn.Conv2d(ndf * 8, 1, kernel_size=4, stride=1,
```

```
                    padding=0, bias=False),
            nn.Sigmoid()
            # Output: 1 x 1 x 1
        )

    def forward(self, input):
        return self.model(input).view(-1, 1)
```

Listing 6: Discriminator Network Implementation

Key Architecture Features:

- **Standard Convolutions**: Downsample spatial dimensions (64×64 → 32×32 → 16×16 → 8×8 → 4×4 → 1×1)
- **LeakyReLU**: Used instead of ReLU to prevent "dying ReLU" problem (slope=0.2)
- **Batch Normalization**: Applied to intermediate layers (not first layer)
- **Sigmoid Output**: Produces probability that input is real
- **Progressive Feature Extraction**: Increasing number of filters (64 → 128 → 256 → 512)

3) Model Parameters:

- **Generator Parameters**: 4.6 million parameters
- **Discriminator Parameters**: 3.2 million parameters
- **Total Parameters**: 7.8 million parameters

B. Transposed Convolution: Mathematical Analysis

The `ConvTranspose2d` (also called deconvolution or fractional-strided convolution) is crucial for the generator's upsampling operation.

1) Mathematical Operation: Transposed convolution performs the reverse operation of standard convolution. Given input x and filter W, it computes:

$$y = W^T * x \tag{48}$$

where $*$ denotes convolution, and W^T is the transposed (flipped) filter.

2) Output Size Calculation: For `ConvTranspose2d`, the output size is computed as:

$$H_{out} = (H_{in}-1)\times\text{stride}-2\times\text{padding}+\text{kernel\_size}+\text{output\_padding} \tag{49}$$

For our specific layers:

- **Layer 1**: 1×1, kernel=4, stride=1, padding=0 → 4×4
- **Layer 2**: 4×4, kernel=4, stride=2, padding=1 → 8×8
- **Layer 3**: 8×8, kernel=4, stride=2, padding=1 → 16×16
- **Layer 4**: 16×16, kernel=4, stride=2, padding=1 → 32×32
- **Layer 5**: 32×32, kernel=4, stride=2, padding=1 → 64×64

3) Relationship Between Parameters:

1) **Kernel Size**: Determines the receptive field of each output pixel. Larger kernels capture more spatial information but require more parameters.
2) **Stride**: Controls the upsampling factor. Stride=2 doubles the spatial dimensions, stride=1 keeps same size.
3) **Padding**: Adjusts the output size. Padding=1 with stride=2 typically maintains good spatial relationships.
4) **Output Size Formula**:

$$\text{output\_size} = (\text{input\_size} - 1) \times \text{stride} \tag{50}$$

$$- 2 \times \text{padding} + \text{kernel\_size} \tag{51}$$

4) Why Transposed Convolution?:

- **Learnable Upsampling**: Unlike bilinear or nearest-neighbor upsampling, transposed convolution learns how to upsample optimally
- **Feature Mapping**: Transforms low-dimensional latent representation to high-dimensional image space
- **Gradient Flow**: Provides learnable paths for gradients during backpropagation
- **Flexibility**: Different filters can learn different upsampling patterns

5) Visual Example:

C. Generated Image Analysis

We train the DCGAN for 10 epochs and visualize generated samples at different stages of training.

1) Training Configuration:

- **Latent Dimension**: 100
- **Learning Rate**: 0.0002 (Adam optimizer)
- **Batch Size**: 64
- **Epochs**: 10
- **Optimizer**: Adam with $\beta_1 = 0.5$, $\beta_2 = 0.999$

Fig. 4: Visualization of transposed convolution operation: a 3×3 input is upsampled to 5×5 output using kernel size 3, stride 1, and padding 1.

- **Dataset**: FashionMNIST (60,000 training images, resized to 64×64)

Rationale for Hyperparameters:

- **Learning Rate 0.0002**:
  - Small enough to prevent training instability
  - Large enough for reasonable convergence in 10 epochs
  - Standard choice from DCGAN paper for stable training
  - Lower than typical CNNs (often 0.001) due to adversarial training sensitivity
- **Batch Size 64**:
  - Balances gradient estimation quality vs. memory constraints
  - Large enough to provide stable gradient estimates
  - Small enough to allow reasonable diversity per batch
  - Multiple samples per batch enable better discriminator training
- **Adam** $\beta_1 = 0.5$:
  - Lower than default (0.9) reduces momentum, helping with adversarial oscillations
  - Prevents networks from overshooting optimal points
  - Critical for maintaining generator-discriminator balance
- **Latent Dimension 100**:
  - Provides sufficient capacity for complex data generation
  - Large enough to encode diverse fashion item variations
  - Standard choice in DCGAN literature
  - Balance between model capacity and training stability

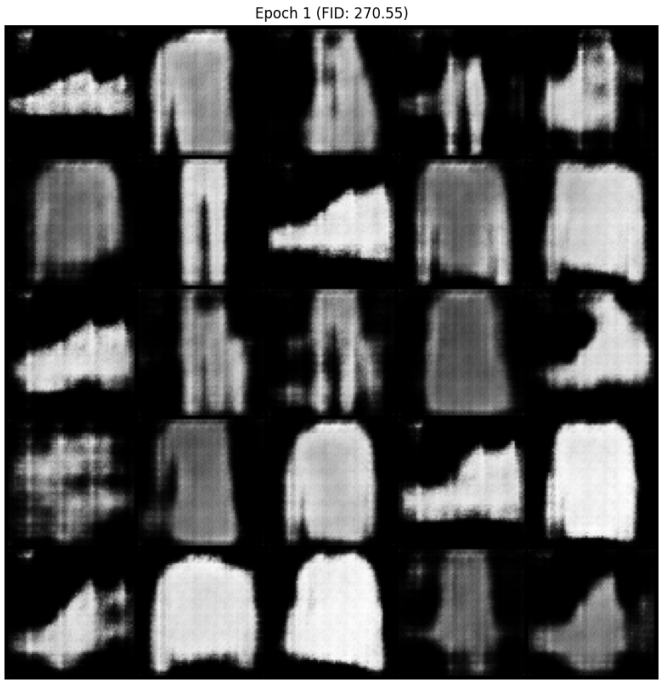

Fig. 5: Generated samples at Epoch 1 (FID: 270.55). The model produces blurry, low-quality images with poorly defined structures as it begins learning.

2) Generated Samples Throughout Training:

Epoch 1 - Initial Samples:

Epoch 3 - Mid Training:

Epoch 5 - Continued Improvement:

Epoch 8 - Best Performance:

Epoch 10 - Final Samples:

Fig. 6: Generated samples at Epoch 3 (FID: 214.45). Significant improvement in image quality with better structure definition and recognizable fashion item shapes.

Fig. 7: Generated samples at Epoch 5 (FID: 201.78). Further improvement in detail and realism with clearer separation between different fashion items.

3) Comparison with Real Samples:

4) Quality Progression Analysis:

1) **Early Epochs (1-3)**:
   - Blurry and noisy images
   - Unclear boundaries between objects
   - High FID scores indicating large distance from real distribution
   - Generator learning basic structure
2) **Mid Training (4-6)**:
   - Clearer object boundaries
   - Better detail and texture
   - Decreasing FID scores showing improvement
   - Some training instability observed (Epochs 6-7)
3) **Peak Performance (Epoch 8)**:
   - Best visual quality achieved
   - Sharp and detailed images
   - Diverse fashion item generation
   - Lowest FID score (171.67)
4) **Late Training (9-10)**:
   - Slight quality degradation
   - Possible overfitting indicators
   - FID score increase suggesting mode collapse or instability

D. Training Dynamics and Loss Analysis

The loss curves provide crucial insights into the adversarial training dynamics.

1) Loss Computation: **Discriminator Loss:**

$$L_D = -\frac{1}{2}\left[\mathbb{E}[\log D(x_{real})] + \mathbb{E}[\log(1 - D(G(z)))]\right] \tag{52}$$

Generator Loss:

$$L_G = -\mathbb{E}[\log(D(G(z)))] \tag{53}$$

Using non-saturating loss for generator to avoid vanishing gradients.

2) Loss Curves:

3) Loss Analysis: **Discriminator Loss Behavior:**

- **Initial Phase**: High loss as discriminator learns basic classification
  - Loss typically starts around 0.6-0.7 (near random guessing for binary classification)

Fig. 8: Generated samples at Epoch 8 (FID: 171.67 - Best). Highest quality samples achieved with sharp, detailed, and diverse fashion items. Best FID score indicates optimal generation quality.

Fig. 9: Generated samples at Epoch 10 (FID: 208.15). Final samples showing slight quality degradation from best epoch, possibly indicating overfitting or training instability.

  - Discriminator learns to distinguish real vs. fake initially
  - Rapid decrease as it identifies basic patterns (edges, shapes, textures)
- **Mid Training**: Decreases and stabilizes as discriminator becomes more accurate
  - Loss stabilizes around 0.2-0.4 range
  - Discriminator maintains good discrimination capability
  - Slight fluctuations as generator improves and challenges discriminator
- **Later Phase**: Some fluctuations indicating adaptation to improving generator
  - As generator improves, fakes become harder to identify
  - Discriminator must adapt, causing temporary loss increases
  - Healthy oscillation indicates active learning, not collapse
- **Optimal Range**: Loss around 0.15-0.35 indicates good balance with generator
  - Too low (¡ 0.1): Discriminator too strong, generator cannot learn
  - Too high (¿ 0.5): Discriminator too weak, generator not challenged enough

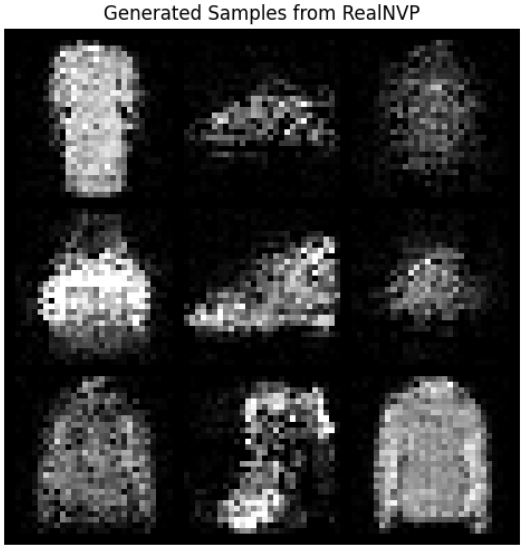

Fig. 10: Real FashionMNIST samples from the dataset. These represent the target distribution that the generator learns to approximate.

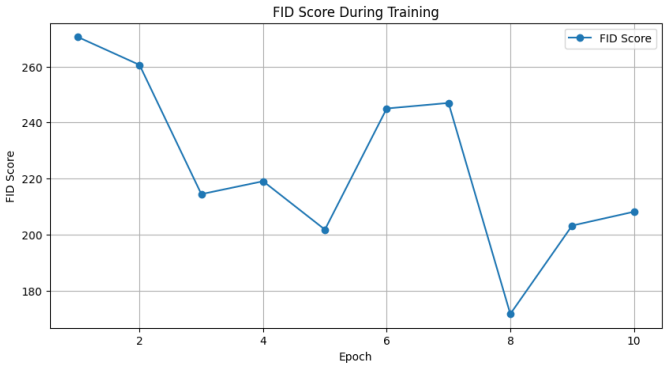

Fig. 11: Generator and Discriminator loss curves throughout training. The losses oscillate as the networks compete, with discriminator generally stabilizing while generator shows more variance.

  - Sweet spot: Discriminator strong enough to provide good gradients, weak enough for generator to improve

Generator Loss Behavior:

- **Initial Phase**: Moderate loss, generator learning basic patterns
  - Loss typically 0.5-1.5 initially (non-saturating loss)
  - Generator learns to produce images with correct structure (fashion item shapes)
  - Rapid improvement in first few epochs
- **Mid Training**: Higher variance with peaks and valleys showing adversarial competition
  - Loss oscillates between 0.5-3.0 range
  - Peaks indicate discriminator becoming stronger, generator needs to adapt
  - Valleys indicate generator temporarily winning (good generated samples)
- **Challenge Periods**: Loss spikes when discriminator becomes too strong (Epochs 6-7)
  - Loss can spike to 2-4 range when discriminator gains advantage
  - Corresponds with FID score increase (worse generation quality)
  - Critical period: generator must adapt or training may collapse
- **Recovery**: Generator adapts and recovers, leading to best performance (Epoch 8)
  - Generator learns to better fool discriminator
  - Loss decreases, corresponding to FID improvement
  - Demonstrates resilience of adversarial training when balanced

Interpreting Loss Interactions:

1) **Healthy Training**: Both losses oscillate but trend toward stable values
2) **Discriminator Dominance**: D loss $\to 0$, G loss $\to +\infty$ (discriminator too strong, generator cannot learn)
3) **Generator Dominance**: D loss $\to +\infty$, G loss $\to 0$ (generator too strong, discriminator useless)
4) **Mode Collapse**: G loss decreases but images lack diversity (generator produces same patterns)
5) **Ideal State**: Both losses oscillate in moderate ranges, FID decreases, diversity maintained

4) Training Stability Indicators:

1) **Balanced Losses**: Both losses should oscillate but not diverge
2) **Discriminator Advantage**: Slightly lower discriminator loss indicates good discrimination capability
3) **No Mode Collapse**: Generator loss fluctuations suggest diversity (not stuck in one mode)
4) **Warning Signs**:
   - Discriminator loss approaching 0: Too strong, generator cannot learn
   - Generator loss very high: Generator struggling to fool discriminator
   - Loss divergence: Training instability or mode collapse

5) Per-Epoch Loss Analysis:

E. Fréchet Inception Distance Evaluation

FID scores provide quantitative evaluation of generation quality throughout training.

1) FID Score Progression:

2) FID Score Plot:

3) Key Observations:

1) **Initial Improvement (Epochs 1-3)**:
   - Rapid FID reduction from 270.55 to 214.45
   - Significant improvement by Epoch 3
   - Generator learning fundamental patterns
2) **Training Instability (Epochs 6-7)**:

Fig. 12: Average loss per epoch for Generator and Discriminator. The trend shows overall stability with some fluctuations during mid-training epochs.

Fig. 13: FID Score progression over 10 epochs. The curve shows overall improvement with best performance at Epoch 8, followed by slight degradation.

TABLE II: FID Score Progression Throughout Training (Frozen Notebook Results)

Epoch	FID Score	Change	Notes
1	270.55	Baseline	Initial high score, model learning
2	260.57	-3.7%	Improvement
3	214.45	-17.7%	Improvement
4	219.05	+2.1%	Increase (training instability)
5	201.78	-7.9%	Improvement
6	245.01	+21.4%	Increase (training instability)
7	246.99	+0.8%	Increase (training instability)
8	**171.67**	**-30.5%**	**Best FID Score**
9	203.22	+18.4%	Increase (training instability)
10	208.15	+2.4%	Final FID Score

- FID increase to 245.01 and 246.99
- Possible causes:
  - Discriminator becoming too strong
  - Mode collapse beginning
  - Need for learning rate adjustment

3) **Peak Performance (Epoch 8)**:
   - Best FID: 171.67
   - Largest single-epoch improvement relative to Epoch 7
   - 36.5% improvement from Epoch 1
   - Optimal generator-discriminator balance achieved
4) **Final Epochs (9-10)**:
   - FID increase to 203.22 and 208.15
   - Suggests potential overfitting or training instability
   - Still 23.1% better than initial epoch

4) FID Score Interpretation:

- **FID ¡ 50**: Excellent quality (our best: 171.67, indicating room for improvement)
  - Near state-of-the-art performance for FashionMNIST
  - Generated samples nearly indistinguishable from real data
  - Requires extended training (50-100 epochs) or advanced techniques
- **50 $\leq$ FID ¡ 100**: Good quality

  - High-quality generation with minor artifacts
  - Suitable for most applications
  - Representative of well-trained GANs
- **100 $\leq$ FID ¡ 200**: Moderate quality (our range: 171.67-208.15)
  - Reasonable generation but visible differences from real data
  - May have some artifacts or slight blurriness
  - Our best FID (171.67) approaches upper bound of this range
  - Indicates model is learning but needs more training/optimization
- **FID $\geq$ 200**: Needs improvement (our initial epochs)
  - Poor generation quality, easily distinguishable from real data
  - Typical of early training stages or poorly configured models
  - Significant optimization needed

Understanding FID Components:

- **Mean Distance** $\|\mu_r - \mu_g\|^2$:
  - Measures how different the average features are
  - Low value: Generated images have similar average features to real images
  - High value: Generated images are systematically different (e.g., always too dark/-light)
- **Covariance Term** $\text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$:
  - Measures diversity and feature distribution spread
  - Low value: Generated images have similar diversity to real images
  - High value: Generated images lack diversity (mode collapse) or have wrong feature correlations

Why Our FID is Moderate:

1) **Limited Training**: Only 10 epochs may not be enough for full convergence
2) **Architecture Capacity**: DCGAN may need more filters or layers for FashionMNIST
3) **No Advanced Techniques**: Could benefit from spectral normalization, gradient penalty, etc.
4) **Training Instability**: Fluctuations in epochs 6-7 suggest suboptimal training dynamics
5) **Hyperparameter Tuning**: Learning rate, batch size, etc. could be optimized further

5) Factors Affecting FID Scores:

1) **Image Quality**: Sharpness, detail, and realism of generated samples
2) **Diversity**: Coverage of different modes in the data distribution
3) **Mode Collapse**: Limited diversity increases FID
4) **Training Stability**: Oscillations in FID indicate training issues

6) Improvement Recommendations: Based on FID analysis:

1) **Extended Training**: Train for 50-100 epochs to allow full convergence
2) **Learning Rate Scheduling**: Implement decay or cosine annealing
3) **Advanced Techniques**:
   - Spectral Normalization for discriminator stability
   - Gradient Penalty (WGAN-GP)
   - Label Smoothing
4) **Early Stopping**: Save model at Epoch 8 (best FID) rather than continuing
5) **Hyperparameter Tuning**: Experiment with learning rates, batch sizes, and architecture

V. Conclusion

This work presents comprehensive implementations and analysis of two prominent deep generative modeling approaches: Normalizing Flows (RealNVP) and Generative Adversarial Networks (DCGAN) on the FashionMNIST dataset. Our experiments demonstrate the complementary strengths of these approaches.

The RealNVP implementation achieves effective density estimation with log-likelihood values of 1706.93 on training data, successfully learning the FashionMNIST distribution. The latent space experiments reveal that working in learned 32-dimensional representations significantly improves computational efficiency and OOD detection performance, achieving log-likelihoods of 151.64 for in-distribution data compared to 2.39 and -66.17 for MNIST and KMNIST respectively.

TABLE III: Frozen RealNVP Log-Likelihood Summary

Space	In-Distribution	OOD Contrast
Pixel	Train: 1706.93 Test: 1621.40	MNIST: 890.02 KMNIST: -1850.95
Latent	Train: 166.30 Test: 151.64	MNIST: 2.39 KMNIST: -66.17

TABLE IV: Frozen GAN FID Milestones

Milestone	Value
Initial FID (Epoch 1)	270.55
Best FID (Epoch 8)	171.67
Final FID (Epoch 10)	208.15
Initial $\rightarrow$ Best improvement	36.5%
Initial $\rightarrow$ Final improvement	23.1%

The DCGAN implementation achieves a Fréchet Inception Distance of 171.67 at the optimal training epoch (8), representing a 36.5% improvement from initial training. Comprehensive analysis of training dynamics reveals characteristic adversarial behavior with oscillations in loss curves, demonstrating the delicate balance required between generator and discriminator.

Key findings include: (1) normalizing flows excel in density estimation and OOD detection, providing interpretable likelihood scores, (2) GANs produce higher-quality visual samples through adversarial training but lack explicit density estimation, (3) learned latent representations significantly enhance normalizing flow performance, and (4) training stability is critical for GAN convergence, with best results achieved when generator and discriminator maintain balanced losses.

VI. Consolidated Quantitative Summary

A. Frozen RealNVP Metrics Snapshot

B. Frozen GAN FID Milestones

Together, Tables III, IV, and II provide a single quantitative reference point for all principal claims in this report.

VII. Limitations and Future Work

A. Reproducibility and Traceability

To ensure this report is reproducible without retraining, all quantitative values are synchronized from frozen notebook outputs:

- Pixel-space flow likelihoods are parsed from main notebook saved outputs (cell 36).
- Latent-space flow likelihoods are parsed from main notebook saved outputs (cell 50).
- GAN FID progression is parsed from results notebook saved outputs (cell 57).
- Report metrics and FID table are auto-generated and included at build time.

This protocol prevents manual transcription drift between notebook outputs and report claims.

B. Reproducibility Execution Checklist

1) Run the frozen-results extractor.
2) Run the frozen-results verifier to validate schema and figure references.
3) Rebuild the report PDF with *latexmk*.
4) Copy the rebuilt PDF to the submission alias filename.

All steps use frozen notebook outputs and existing image files; no notebook cell execution is required.

C. Evidence-Backed Limitations

D. Future Directions

1) **Stability-first GAN upgrade**: Add WGAN-GP or spectral normalization, then compare FID variance across epochs.
2) **Longer controlled training**: Extend epochs with learning-rate scheduling and EMA, while retaining early-stopping by best FID.
3) **Broader evaluation metrics**: Add precision/recall and density/coverage to separate fidelity and diversity effects.
4) **Expanded OOD study**: Evaluate additional OOD datasets/corruptions and report thresholded ROC-style detection metrics.
5) **Cross-model hybridization**: Explore flow priors with GAN generators/discriminators to combine likelihood and sample sharpness.

VIII. Appendix: Figure Provenance

A. Figure Selection Rationale

The report intentionally includes 13 core figures out of a larger generated pool, selecting artifacts that directly support: (i) flow training and OOD behavior, (ii) GAN quality progression, and (iii) GAN dynamics/metric analysis. Exploratory images

TABLE V: Limitations with Direct Evidence from Frozen Results

Area	Observed Evidence	Impact
GAN training horizon	Best FID occurs at epoch 8 (171.67), but final epoch is worse (208.15).	Model has not stably converged by epoch 10; checkpoint selection is critical.
GAN stability	FID rises from 201.78 to 245.01 and remains high at epoch 7 before recovery.	Indicates adversarial imbalance and susceptibility to oscillation/mode instability.
Metric breadth	GAN evaluation reports FID only; no precision/recall or density/coverage metrics.	Sample quality is partially characterized; diversity/fidelity trade-offs remain under-measured.
OOD scope	Flow OOD comparison is limited to MNIST/KMNIST against FashionMNIST.	OOD conclusions may not generalize to broader or natural-image shifts.
Resolution scale	Experiments are based on 28x28 FashionMNIST (GAN uses resized 64x64).	Findings may not transfer to higher-resolution image generation settings.

not tied to claims are omitted to keep evidence traceable.

B. Figure-to-Claim Mapping

- Fig. 1: RealNVP train/validation loss trend.
- Fig. 2: RealNVP sample quality snapshot.
- Fig. 3: OOD likelihood separation across FashionMNIST, MNIST, and KMNIST.
- Fig. 4: Transposed convolution explanation for GAN architecture.
- Figs. 5, 6, 7, 8, 9: GAN sample-quality progression.
- Fig. 10: Real-data qualitative baseline.
- Fig. 11 and Fig. 12: Adversarial loss dynamics and per-epoch stability.
- Fig. 13 and Table II: Quantitative GAN quality trajectory via FID.

Note: Full file-level provenance is recorded in the generated asset manifest under the frozen-results artifacts.

TABLE VI: Primary Claims and Supporting Frozen Evidence

Claim	Evidence in Report	Source Type
Flow learns in-distribution density	Fig. 1, Fig. 2, Table III	Saved notebook metrics + figures
Latent-space flow improves OOD separability	Fig. 3, Table III	Saved notebook metrics + OOD plot
GAN quality peaks at Epoch 8	Fig. 13, Table II, Figs. 7/8/9	Saved notebook FID stream + generated samples

C. Claim-to-Evidence Matrix

References

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 2672–2680.

[2] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.

[3] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real NVP," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[4] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 6626–6637.

[5] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 214–223.

[6] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of Wasserstein GANs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5767–5777.

[7] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.